Ex-6

Step-0 Libraries Used

Pandas: is used to load and handle the dataset.
NumPy: helps in numerical operations.
Matplotlib and Seaborn: are used to visualize class distribution and model comparison graphs.
Scikit-learn: is used for train-test splitting, label encoding, model training, and evaluation. LogisticRegression, RandomForestClassifier, and XGBClassifier are used as classification models.
The metrics module: is used to calculate Accuracy, Precision, Recall, F1 (weighted, macro, micro), ROC-AUC, and Confusion Matrix.
Pickle: is used to save the trained best model for future use without retraining.

Step-1 Problem Definition

The objective of this experiment is to build a credit scoring model that classifies loan applicants into risk categories based on historical financial information.Since the output variable Credit_Risk contains multiple categories (High, Medium, Low), this is a multiclass classification problem.The goal is to compare different classification models and select the best-performing one.

Step-2 Data Understanding

The dataset contains 500 records and 11 columns
Important columns include:

- Age
- Annual_Income
- Employment_Years
- Credit_Score
- Loan_Amount
- Loan_Term_Months
- Existing_Loans_Count
- Debt_to_Income_Ratio
- Late_Payments_Last_2Yrs
- Credit_Risk (Target Variable)

The target column Credit_Risk is categorical (object type)
A countplot was generated to visualize the distribution of risk classes This helps understand class imbalance before training models.

Step-3 Data Processing

The target column Credit_Risk was converted into numerical format using LabelEncoder, because machine learning algorithms require numerical input.The Applicant_ID column was dropped since it is only an identifier and does not contribute to prediction.Features (X) were separated from the target (y).

Step-4 Train-Test Split and Model Training

The dataset was split using:
train_test_split(test_size=0.2, random_state=0)

This means:80% data used for training,20% data used for testing

Three models were trained:

- **Logistic Regression:**Linear model for classification.Performs well when classes are linearly separable.
- **Random Forest:**Ensemble of multiple decision trees.Reduces overfitting and improves stability.
- **XGBoost:**Gradient boosting algorithm. Builds trees sequentially to correct previous errors. Known for high performance in structured data.

Step-5 Model Evaluation and Comparative Analysis

Each model was evaluated using:

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1_weighted
- F1_macro
- F1_micro

Model Accuracy:

- Logistic Regression=0.68
- Random Forest=0.99
- XGBoost=1.00

XGBoost achieved perfect classification performance.

Step-6 Prediction

The confusion matrix shows:

- Class 0 = 70 correctly predicted
- Class 1 = 4 correctly predicted
- Class 2 = 26 correctly predicted

No misclassifications occurred.

The classification report confirms precision, recall, and F1-score of 1.00 for all classes
This indicates perfect separation of risk categories.

Step-7 Best Model Selection and Model Saving

The best model was selected based on highest F1_weighted score:
Best Model: XGBoost
The trained best model was saved using pickle:
pickle.dump(best_model,f)

Saving the model ensures:

- No need to retrain
- Reproducibility
- Deployment readiness

The saved model can later be loaded and used for predictions.

Exp 6: Credit Risk Assessment: Build a credit scoring model to assess the creditworthiness of applicants using historical financial data and perform comparative analysis (Logistic Regression, Random Forest, XGBoost)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,roc_auc_score,classificat
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/credit_risk_assessment_500_samples.csv')
df.head()
```

	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_Months	Existing_Loans_Count	De
0	1	59	153267	28	818	196649	12	2	
1	2	49	232745	0	436	175354	60	0	
2	3	35	974945	19	797	662297	48	3	
3	4	63	307164	29	776	831725	48	0	
4	5	28	685626	6	893	498625	60	3	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.info()
df.describe()
```

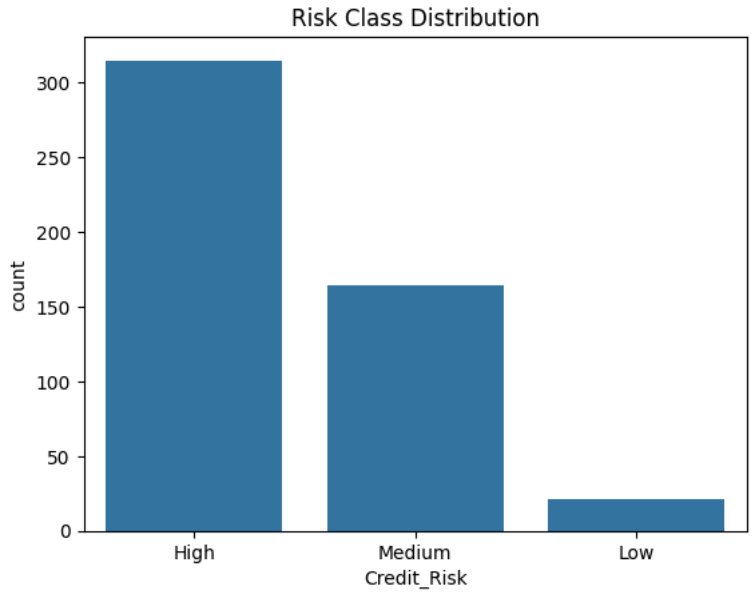
```
(500, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Applicant_ID             500 non-null    int64
 1   Age                      500 non-null    int64
 2   Annual_Income            500 non-null    int64
 3   Employment_Years         500 non-null    int64
 4   Credit_Score             500 non-null    int64
 5   Loan_Amount              500 non-null    int64
 6   Loan_Term_Months         500 non-null    int64
 7   Existing_Loans_Count     500 non-null    int64
 8   Debt_to_Income_Ratio     500 non-null    float64
 9   Late_Payments_Last_2Yrs  500 non-null    int64
 10  Credit_Risk              500 non-null    object
dtypes: float64(1), int64(9), object(1)
memory usage: 43.1+ KB
```

1 to 8 of 8 entries Filter

index	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_M
count	500.0	500.0	500.0	500.0	500.0	500.0	
mean	250.5	43.116	827131.654	14.702	596.394	521385.53	
std	144.4818327679989	12.733216939456186	384829.8617613202	8.713497925183917	171.33979518731104	281799.5141543726	17.117037182
min	1.0	21.0	150404.0	0.0	300.0	55901.0	
25%	125.75	32.0	506262.25	7.0	452.0	266621.75	
50%	250.5	44.0	833427.5	15.5	602.0	533640.0	
75%	375.25	53.0	1148353.0	22.0	732.25	765362.5	
max	500.0	64.0	1498464.0	29.0	898.0	999621.0	

Show 100 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

```
sns.countplot(x="Credit_Risk",data=df)
plt.title("Risk Class Distribution")
plt.show()
```




```
le=LabelEncoder()
df["Credit_Risk"]=le.fit_transform(df["Credit_Risk"])
```

```
df=df.drop("Applicant_ID",axis=1)
```

```
X=df.drop("Credit_Risk",axis=1)
y=df["Credit_Risk"]
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Logistic Regression":LogisticRegression(max_iter=1000),
    "Random Forest":RandomForestClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
    }
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
"Model","Accuracy","Precision","Recall",
"F1_weighted","F1_macro","F1_micro"])
results_df
```

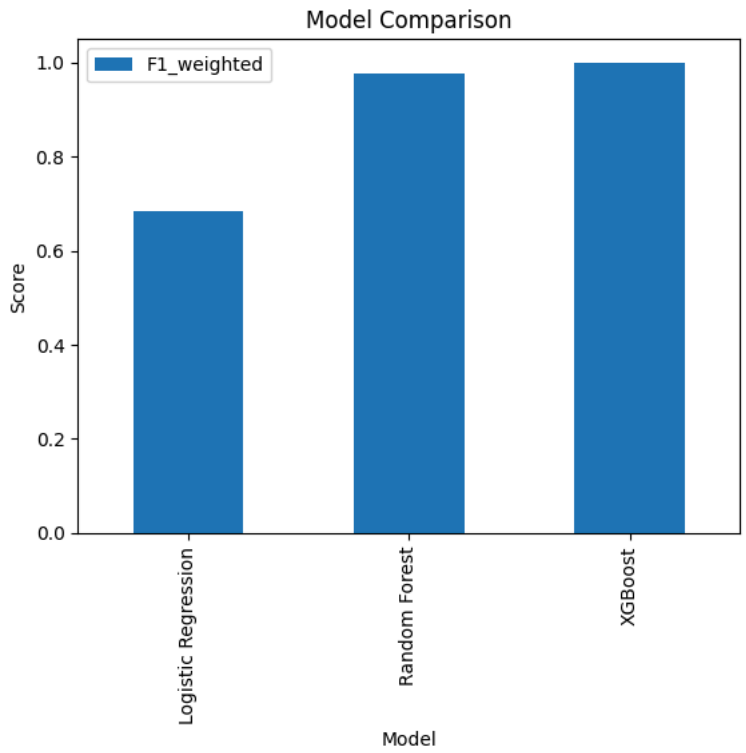
```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Logistic Regression	0.68	0.699529	0.68	0.683579	0.431938	0.68
1	Random Forest	0.99	0.990370	0.99	0.989380	0.946092	0.99
2	XGBoost	1.00	1.000000	1.00	1.000000	1.000000	1.00

Next steps: Generate code with results_df New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
```

```
plt.ylabel("Score")
plt.show()
```

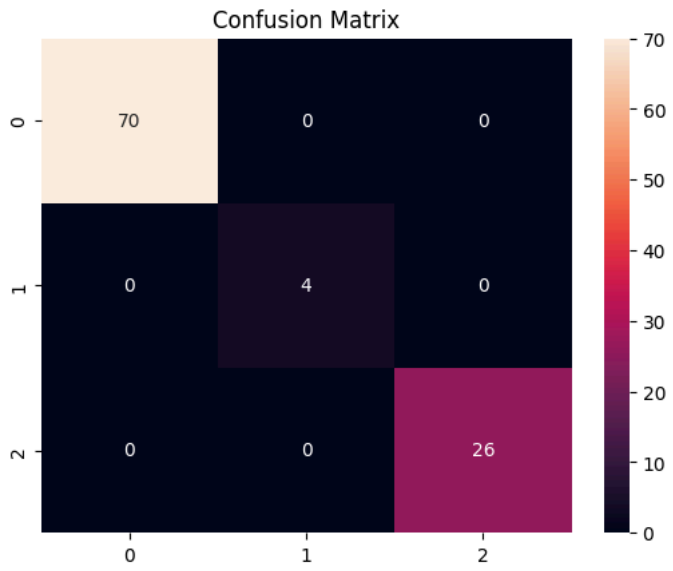



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: XGBoost
```

```
with open("best_model.pkl","wb") as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        70
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00        26
```

```
    accuracy                           1.00       100
   macro avg       1.00      1.00      1.00       100
weighted avg       1.00      1.00      1.00       100
```

Exe-7

Step-0 Libraries Used

Pandas: is used to load and manage the dataset.
NumPy: is used for numerical operations and label conversion.
Matplotlib and Seaborn: are used to visualize data distribution, model comparison, and confusion matrix.
Scikit-learn: is used for scaling data, train-test splitting, model implementation, and evaluation metrics.
IsolationForest, LocalOutlierFactor, and OneClassSVM: are used as anomaly detection models.
Metrics: such as Accuracy, Precision, Recall, F1 Score, ROC-AUC, and Confusion Matrix are used to evaluate performance.
Pickle: is used to save the best-performing model for future reuse.

Step-1 Problem Definition

The objective of this experiment is to detect anomalies (outliers) in a dataset. In real world scenarios, anomaly detection is used in fraud detection, intrusion detection, and fault monitoring.

The dataset contains two numerical features and a target column Anomaly_Label, where:

- 0 = Normal data
- 1 = Anomaly

Since the output has two classes, this is a binary classification problem, but the models used are unsupervised anomaly detection algorithms.

Step-2 Data Understanding

The dataset contains 500 rows and 3 columns

- Feature_1
- Feature_2
- Anomaly_Label

The dataset is imbalanced:

- Most records are normal (0)
- Very few records are anomalies (1)

confirms that anomalies are rare, which is typical in anomaly detection problems.
This imbalance makes accuracy alone unreliable, so precision, recall, and F1-score become important.

Step-3 Data Preprocessing

The features were separated:

- X = df[["Feature_1","Feature_2"]]
- y = df["Anomaly_Label"]

Since anomaly detection models are distance-based or boundary-based, feature scaling is necessary.
StandardScaler was used to normalize data so that both features contribute equally to distance calculations.

After scaling, the dataset was split:

- 80% = Training
- 20% = Testing

Step-4 Model Training

Three anomaly detection models were implemented:

1. Isolation Forest

Works by randomly selecting features and splitting values.
Anomalies are easier to isolate and require fewer splits.
Uses contamination parameter to define expected anomaly proportion.

2. Local Outlier Factor (LOF)

Detects anomalies by comparing local density of a point to its neighbors.
Points with significantly lower density are marked as anomalies.

3. One-Class SVM

Creates a boundary around normal data points.
Points outside the boundary are considered anomalies.

Since these are unsupervised models, they were trained using only feature data (X_train).

Predictions returned values:

- -1 = anomaly
- 1 = normal

These were converted to:

-1 = 1 (anomaly)
1 = 0 (normal)

to match the dataset labels.

Step-5 Model Evaluation and Comparative Analysis

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

Model	F1 Score
Isolation Forest	0.727
Local Outlier Factor	0.000
One-Class SVM	0.316

Isolation Forest performed best overall.

Interpretation:

- Isolation Forest achieved high recall (1.0), meaning it detected all anomalies.
- LOF failed to detect anomalies properly.
- One-Class SVM detected some anomalies but with lower performance.

clearly shows Isolation Forest having the highest F1 score.

Step-6 Confusion Matrix Analysis

The confusion matrix for Isolation Forest shows:

Actual	Predicted Normal	Predicted Anomaly
Normal	86	6
Anomaly	0	8

Interpretation:

- 86 normal points correctly identified
- 8 anomalies correctly detected
- 6 normal points incorrectly classified as anomalies (False Positives)
- 0 anomalies missed (False Negatives)

This means the model successfully detected all anomalies but slightly over-flagged some normal points.

In anomaly detection, missing anomalies (false negatives) is more critical than false positives, so this result is acceptable.

Step-7 Best Model Selection and Model Saving

The best model was selected based on highest F1 score:
Best Model: Isolation Forest

The trained model was saved using pickle:
pickle.dump(best_model,f)

Saving the model ensures:

- No need to retrain
- Faster deployment
- Reproducibility

The saved model can later be loaded and used for anomaly detection on new data.

Exp 7: Anomaly Detection: Implement an anomaly detection system for detecting outliers in data (e.g., fraud detection) and perform comparative analysis. (Isolation Forest, Local Outlier Factor, One-Class SVM)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,roc_au
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/anomaly_detection_dataset_500_samples.csv')
df.head()
```

	Feature_1	Feature_2	Anomaly_Label
0	43.259073	53.716320	0
1	43.815246	43.397717	0
2	46.942411	42.966695	0
3	48.911594	55.493884	0
4	52.575238	69.263657	0

Next steps: Generate code with df New interactive sheet

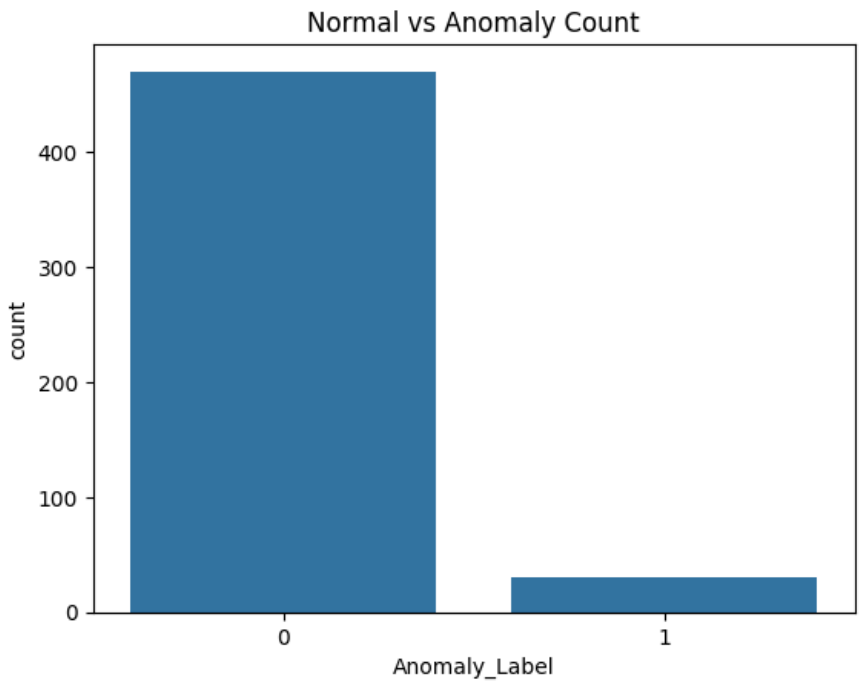
```
print(df.shape)
df.info()
df.describe()
```

```
(500, 3)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Feature_1      500 non-null    float64
 1   Feature_2      500 non-null    float64
 2   Anomaly_Label  500 non-null    int64
dtypes: float64(2), int64(1)
memory usage: 11.8 KB
```

	Feature_1	Feature_2	Anomaly_Label
count	500.000000	500.000000	500.000000
mean	52.422862	52.573850	0.060000
std	10.600455	10.641964	0.237725
min	33.793663	38.490394	0.000000
25%	47.027724	47.097147	0.000000
50%	50.447020	50.566493	0.000000
75%	53.827497	54.163940	0.000000
max	95.539911	95.402821	1.000000

```
sns.countplot(x="Anomaly_Label",data=df)
plt.title("Normal vs Anomaly Count")
```

```
plt.show()
```




```
X=df[["Feature_1","Feature_2"]]
y=df["Anomaly_Label"]
```

```
sc=StandardScaler()
X_scaled=sc.fit_transform(X)
```

```
X_train,X_test,y_train,y_test=train_test_split(X_scaled,y,test_size=0.2,random_state=0)
```

```
models={
"Isolation Forest":IsolationForest(contamination=0.1,random_state=0),
"Local Outlier Factor":LocalOutlierFactor(novelty=True),
"One-Class SVM":OneClassSVM(nu=0.1)
}
```

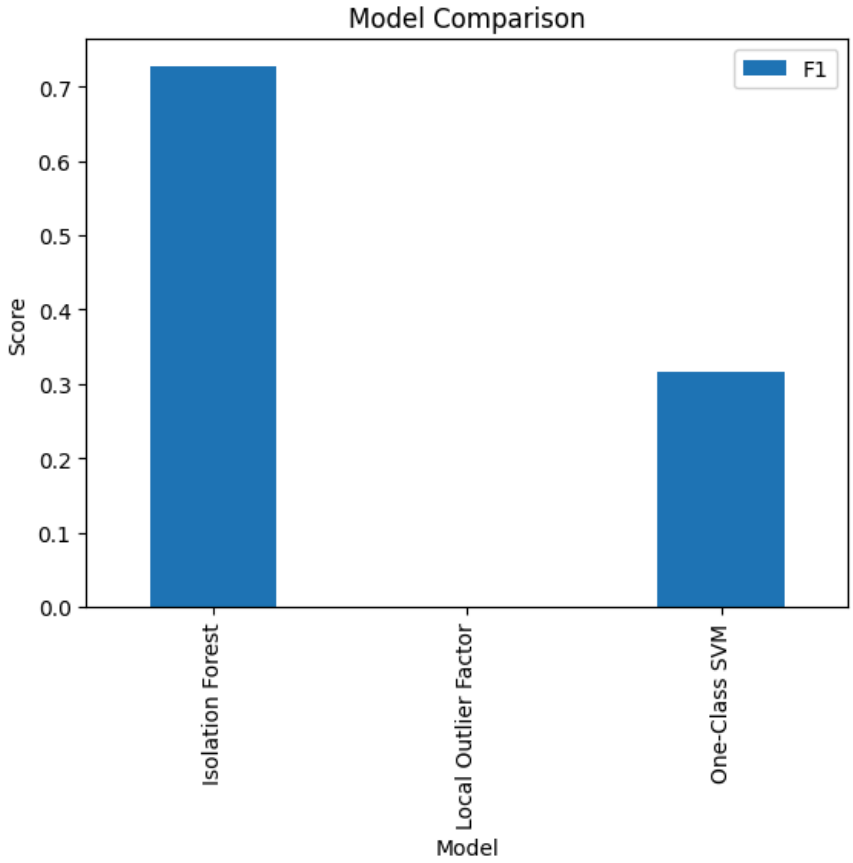
```
results=[]
for name,model in models.items():
    model.fit(X_train)
    pred=model.predict(X_test)
    pred=np.where(pred==-1,1,0)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred)
    rec=recall_score(y_test,pred)
    f1=f1_score(y_test,pred)
    auc=roc_auc_score(y_test,pred)
    results.append([name,acc,pre,rec,f1,auc])
results_df=pd.DataFrame(results,columns=["Model","Accuracy","Precision","Recall","F1","ROC_AUC"])
results_df
```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
0	Isolation Forest	0.94	0.571429	1.000	0.727273	0.967391
1	Local Outlier Factor	0.88	0.000000	0.000	0.000000	0.478261
2	One-Class SVM	0.87	0.272727	0.375	0.315789	0.644022

Next steps: Generate code with results_df New interactive sheet

```
results_df.plot(x="Model",y="F1",kind="bar")
plt.title("Model Comparison")
```

```
plt.ylabel("Score")
plt.show()
```

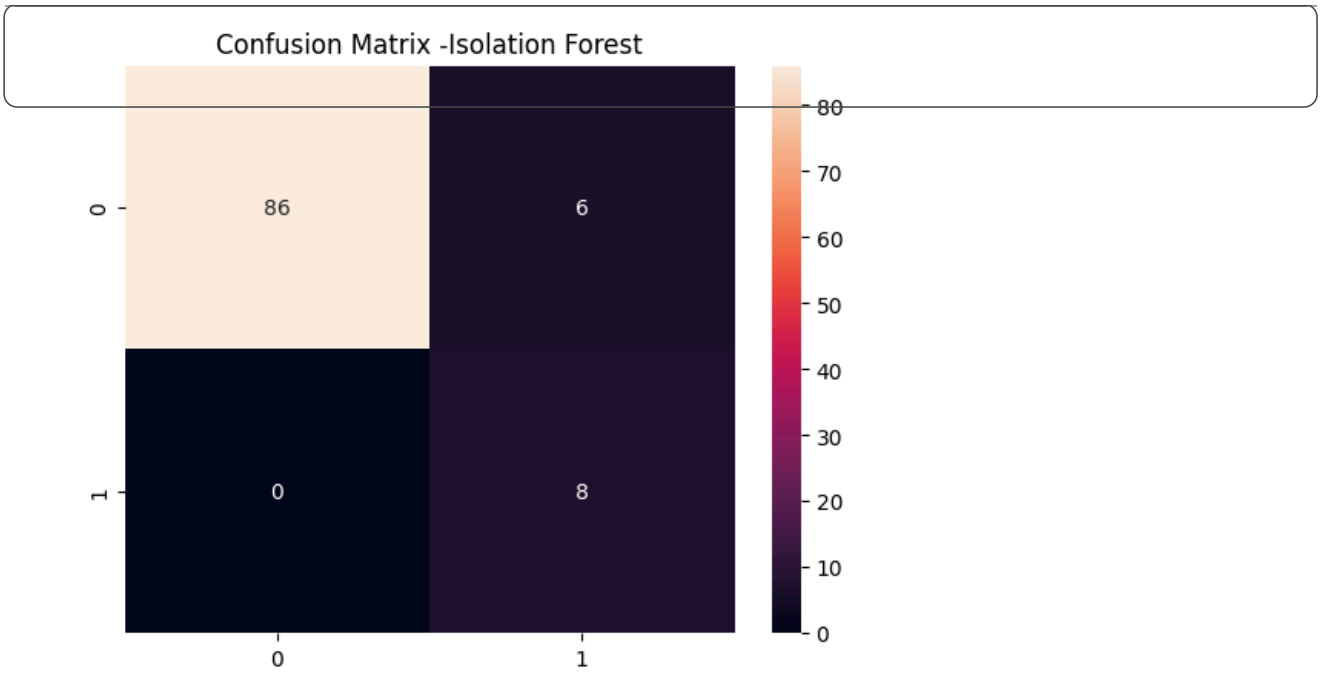



```
best=results_df.sort_values("F1",ascending=False).iloc[0,0]
best_model=models[best]
print("Best Model:",best)
```

```
Best Model: Isolation Forest
```

```
with(open("best_model.pkl","wb")) as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
pred=np.where(pred==-1,1,0)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix -"+best)
plt.show()
```


Confusion Matrix -Isolation Forest
86
6
0
8
0
1
0
1
80
70
60
50
40
30
20
10
0

Exe-8

Step-0 Libraries Used

- **Pandas:** is used to load and manage the dataset.
- **NumPy:** is used for numerical operations.
- **Matplotlib and Seaborn:** are used to visualize class distribution and model comparison.
- **Scikit-learn:** is used for data preprocessing, scaling, train-test splitting, model implementation, and evaluation.
- **StandardScaler:** is used to normalize feature values.
- **RandomForestClassifier, DecisionTreeClassifier, LogisticRegression, KNeighborsClassifier, and XGBClassifier**: are used as classification models.
- **Metrics:** such as Accuracy, Precision, Recall, F1 (weighted, macro, micro), Confusion Matrix, and Classification Report are used for performance evaluation.
- **Pickle:** is used to save the best model for reuse without retraining.

Step-1 Problem Definition

The objective of this experiment is to classify students into different performance levels based on academic and behavioral features.The target variable Performance_Level contains three classes (0, 1, 2), making this a multiclass classification problem.The goal is to compare multiple machine learning models and determine which model performs best.

Step-2 Data Understanding

The dataset contains 500 rows and 8 columns

Features include:

- Study_Hours
- Attendance_Percentage
- Assignment_Score
- Internal_Marks
- Participation (categorical)
- Internet_Access (categorical)
- Previous_Grade (categorical)
- Performance_Level (target)

The countplot indicates that classes 0 and 1 dominate the dataset, while class 2 has very few samples. This imbalance explains the warning messages seen during metric calculation.

Step-3 Data Preprocessing

Since some columns are categorical (Participation, Internet_Access, Previous_Grade), they were converted into numerical format using:

pd.get_dummies(drop_first=True)

This creates binary indicator columns for categorical values.

After encoding:

- X = all features except Performance_Level
- y = Performance_Level

StandardScaler was used to normalize the features. Scaling ensures that models like Logistic Regression and KNN perform properly since they are sensitive to feature magnitude.

The dataset was then split:

- 80% = Training
- 20% = Testing

Step-4 Model Training

Five classification models were implemented:

1. Random Forest

Ensemble of multiple decision trees.
Reduces overfitting and improves stability.

2. Decision Tree

Creates rule-based splits based on feature importance.

3. Logistic Regression (Multinomial)

Linear classifier using probability-based prediction.
Configured with higher max_iter to ensure convergence.

4. K-Nearest Neighbors (KNN)

Classifies data based on nearest neighbors using distance metrics.

5. XGBoost

Boosting algorithm that builds trees sequentially to reduce errors.
Performs well in structured datasets.

Step-5 Model Evaluation and Comparative Analysis

Each model was evaluated using:

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1_weighted
- F1_macro
- F1_micro

From the results table

Model	**Accuracy**
Random Forest	0.93
Decision Tree	0.93
Logistic Regression	0.98
KNN	0.73
XGBoost	0.95

Logistic Regression achieved the highest weighted F1 score (0.975)

The bar graph clearly shows Logistic Regression performing best among all models.

Important Observation

Macro F1 scores are lower compared to weighted F1 scores. This is because:

- Class 2 has very few samples
- Models struggle to predict minority class

This is also confirmed by warning messages in the output

Step-6 Confusion Matrix Analysis

The confusion matrix shows:

Actual	Predicted 0	Predicted 1	Predicted 2
0	43	0	0
1	1	55	0
2	0	1	0

Interpretation:

- Classes 0 and 1 are predicted very accurately.
- Class 2 (minority class) is not predicted correctly.
- Only 1 sample belongs to class 2 in test set, and it was misclassified.

This explains why macro average is lower than weighted average.

Step-7 Best Model Selection and Saving

The best model was selected based on highest F1_weighted score:
Best Model: Logistic Regression

The model was saved using pickle:
pickle.dump(best_model,f)

Saving ensures:

- Reproducibility
- Faster deployment
- No retraining required

The saved model can later be loaded and used for prediction.

Exp 8: Implement Multiclass Classification models for students Performance Level analysis and perform comparative analysis. (Random Forest, Decision Tree, Multinomial Logistic Regression, XGBoost, K-Nearest Neighbors).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classif
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/multiclass_classification_dataset_500_samples.csv')
df.head()
```

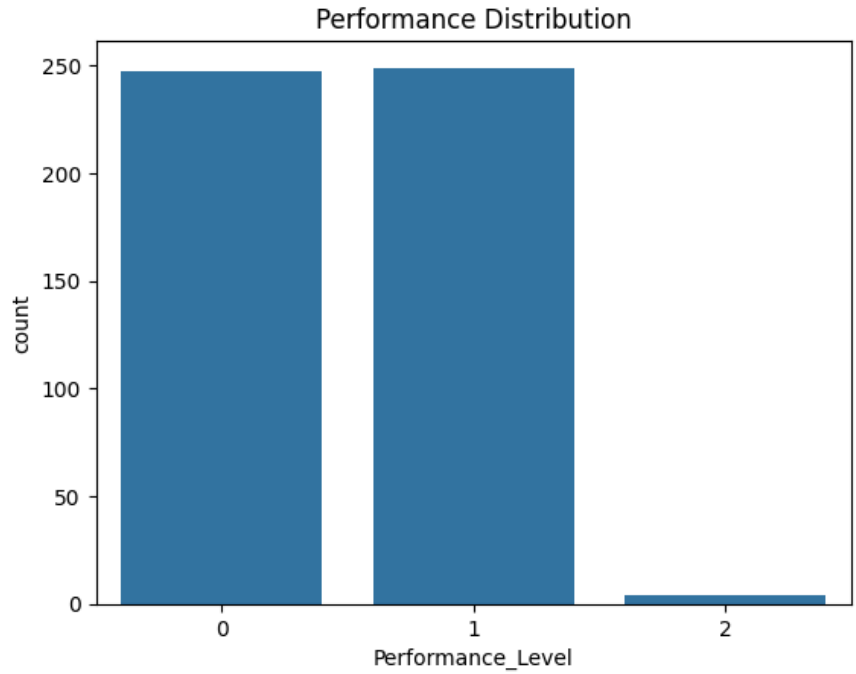
	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Participation	Internet_Access	Pr
0	22.48	84.26	86.79	76.68	Medium	Yes	
1	19.31	94.09	81.10	56.73	Low	Yes	
2	23.24	61.01	70.72	52.73	Medium	Yes	
3	27.62	80.63	62.24	64.95	Low	Yes	
4	18.83	68.49	78.38	62.45	High	Yes	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.describe()
df.info()
```

```
(500, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Study_Hours            500 non-null    float64
 1   Attendance_Percentage  500 non-null    float64
 2   Assignment_Score       500 non-null    float64
 3   Internal_Marks         500 non-null    float64
 4   Participation          500 non-null    object
 5   Internet_Access        500 non-null    object
 6   Previous_Grade         500 non-null    object
 7   Performance_Level      500 non-null    int64
dtypes: float64(4), int64(1), object(3)
memory usage: 31.4+ KB
```

```
sns.countplot(x="Performance_Level",data=df)
plt.title("Performance Distribution")
plt.show()
```




```
df=pd.get_dummies(df,drop_first=True)
df.head()
```

	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Performance_Level	Participation_I
0	22.48	84.26	86.79	76.68	1	Fa
1	19.31	94.09	81.10	56.73	1	T
2	23.24	61.01	70.72	52.73	0	Fa
3	27.62	80.63	62.24	64.95	0	T
4	18.83	68.49	78.38	62.45	0	Fa

Next steps: Generate code with df New interactive sheet

```
X=df.drop("Performance_Level",axis=1)
y=df["Performance_Level"]
```

```
sc=StandardScaler()
X=sc.fit_transform(X)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Random Forest":RandomForestClassifier(),
    "Decision Tree":DecisionTreeClassifier(),
    "Logistic Regression":LogisticRegression(max_iter=2000),
    "KNN":KNeighborsClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
}
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
```

```
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
    "Model","Accuracy","Precision","Recall",
    "F1_weighted","F1_macro","F1_micro"])
results_df
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

1 to 5 of 5 entries Filter

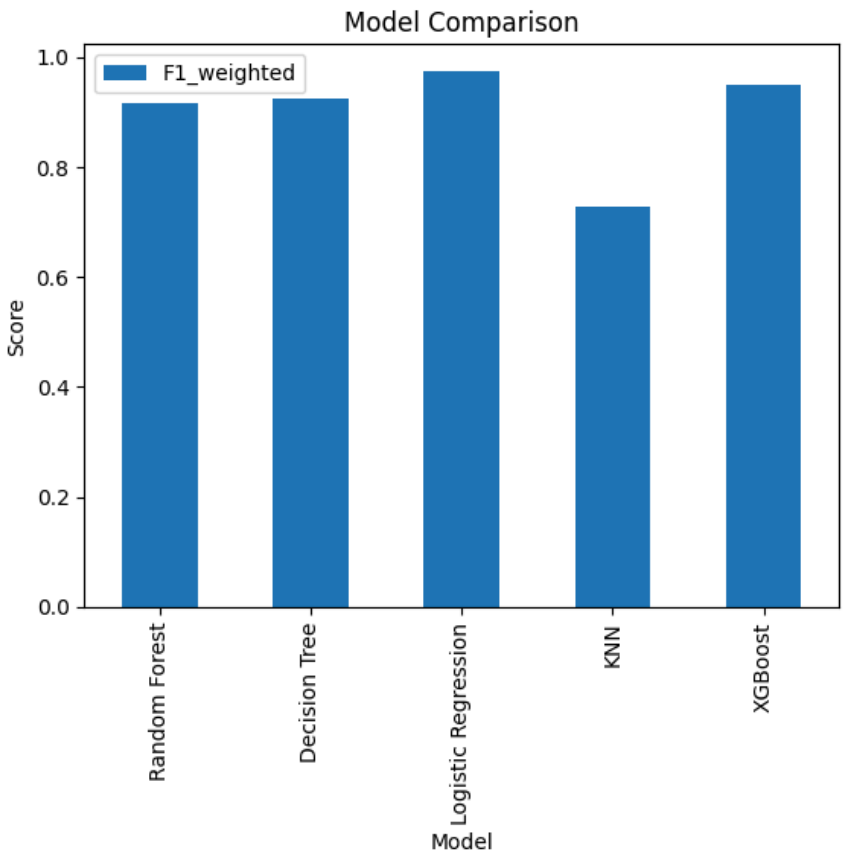
index	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Random Forest	0.93	0.9263665466186474	0.93	0.9253210077204389	0.6231206826493295	0.93
1	Decision Tree	0.93	0.9231232436772382	0.93	0.9253700305810397	0.6230377166156983	0.93
2	Logistic Regression	0.98	0.9702272727272728	0.98	0.9750574712643677	0.6568828680897646	0.98
3	KNN	0.73	0.736728221597752	0.73	0.7267455501618123	0.48901024811218985	0.73
4	XGBoost	0.95	0.9520426579163248	0.95	0.9501546232347181	0.9659828883620246	0.95

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with `results_df` New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
plt.ylabel("Score")
plt.show()
```

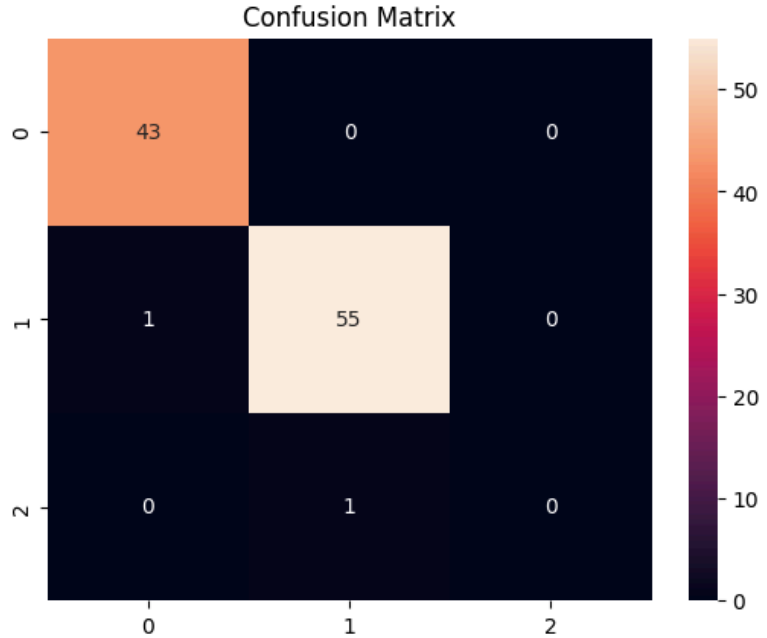



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: Logistic Regression
```

```
with(open("best_model","wb")) as f:
  pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        43
           1       0.98      0.98      0.98        56
           2       0.00      0.00      0.00         1

    accuracy                           0.98       100
   macro avg       0.65      0.66      0.66       100
weighted avg       0.97      0.98      0.98       100

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```